

Experiment 4A — Support Vector Machines (SVM)

Aim

To build an SVM model for binary classification, tune its hyperparameters using Grid Search with cross-validation, and evaluate the model using accuracy, precision, recall, F1-score, confusion matrix and ROC-AUC.

Algorithm

1. Import the required libraries.
2. Load the Pet Adoption dataset.
3. Handle missing values.
4. Convert the adoption-speed target into two classes.
5. Encode categorical variables.
6. Split the dataset into training and testing sets.
7. Standardize the features.
8. Create an SVM model using the RBF kernel.
9. Perform hyperparameter tuning using GridSearchCV.
10. Train the model using the best parameters.
11. Predict the test data.
12. Calculate accuracy, precision, recall, F1-score and ROC-AUC.
13. Display the confusion matrix and ROC curve.

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.svm import SVC
```

```

from sklearn.metrics import (

    accuracy_score,

    precision_score,

    recall_score,

    f1_score,

    confusion_matrix,

    classification_report,

    roc_auc_score,

    roc_curve
)

df = pd.read_csv("pet_adoption.csv")

df["Breed"] = df["Breed"].fillna(df["Breed"].mode()[0])

df["Vaccinated"] = df["Vaccinated"].fillna(df["Vaccinated"].mode()[0])

df["Fee"] = df["Fee"].fillna(df["Fee"].median())

df["AdoptionBinary"] = (df["AdoptionSpeed"] <= 2).astype(int)

X = df.drop(["AdoptionSpeed", "AdoptionBinary"], axis=1)

y = df["AdoptionBinary"]

categorical_cols = X.select_dtypes(

    include=["str", "object"]

).columns

X_encoded = X.copy()

for col in categorical_cols:

    le = LabelEncoder()

```

```

X_encoded[col] = le.fit_transform(
    X_encoded[col].astype(str)
)

X_train, X_test, y_train, y_test = train_test_split(
    X_encoded,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

svm = SVC(
    kernel="rbf",
    probability=True,
    random_state=42
)

param_grid = {
    "C": [0.1, 1, 10, 100],
    "gamma": ["scale", 0.01, 0.001, 0.0001]
}

grid = GridSearchCV(

```

```

    estimator=svm,

    param_grid=param_grid,

    scoring="f1",

    cv=5,

    n_jobs=-1
)

grid.fit(X_train_scaled, y_train)

print("Best Parameters:", grid.best_params_)

print("Best Cross-Validation Score:", round(grid.best_score_, 4))

best_svm = grid.best_estimator_

y_pred = best_svm.predict(X_test_scaled)

y_prob = best_svm.predict_proba(X_test_scaled)[:, 1]

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(y_test, y_pred)

recall = recall_score(y_test, y_pred)

f1 = f1_score(y_test, y_pred)

auc = roc_auc_score(y_test, y_prob)

cm = confusion_matrix(y_test, y_pred)

print("\n===== SVM RESULTS =====")

print("Accuracy :", round(accuracy, 4))

print("Precision:", round(precision, 4))

print("Recall  :", round(recall, 4))

print("F1-Score :", round(f1, 4))

```

```
print("ROC-AUC :", round(auc, 4))

print("\nClassification Report:")

print(classification_report(y_test, y_pred))

plt.figure(figsize=(6, 5))

sns.heatmap(

    cm,

    annot=True,

    fmt="d",

    cmap="Blues",

    cbar=False

)

plt.title("SVM Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()

fpr, tpr, thresholds = roc_curve(

    y_test,

    y_prob

)

plt.figure(figsize=(7, 5))

plt.plot(

    fpr,

    tpr,
```

```
label=f"SVM (AUC = {auc:.3f})"  
  
)  
  
plt.plot(  
  
    [0, 1],  
  
    [0, 1],  
  
    linestyle="--"  
  
)  
  
plt.xlabel("False Positive Rate")  
  
plt.ylabel("True Positive Rate")  
  
plt.title("SVM ROC Curve")  
  
plt.legend()  
  
plt.grid(True)  
  
plt.show()
```

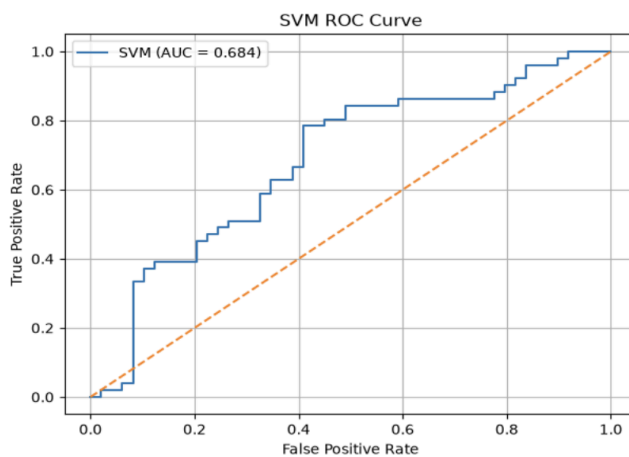
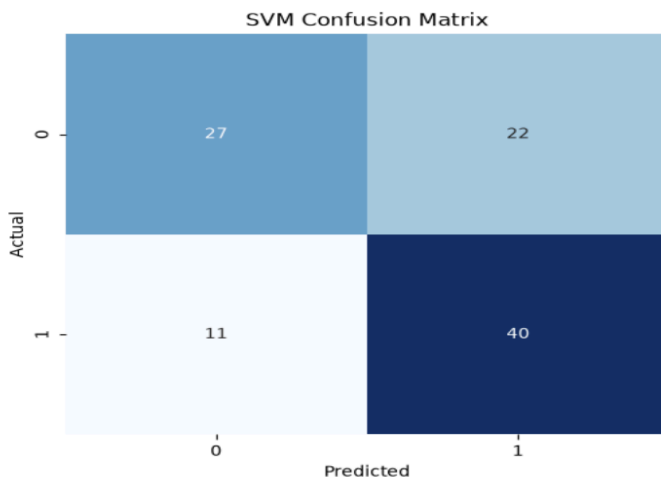
Output:

```
Best Parameters: {'C': 0.1, 'gamma': 'scale'}  
Best Cross-Validation Score: 0.7023
```

```
===== SVM RESULTS =====
```

```
Accuracy : 0.67  
Precision: 0.6452  
Recall : 0.7843  
F1-Score : 0.708  
ROC-AUC : 0.6843
```

```
Classification Report:  
              precision    recall  f1-score   support  
  
     0       0.71        0.55        0.62         49  
     1       0.65        0.78        0.71         51  
  
 accuracy          0.67  
 macro avg         0.68        0.67        0.66         100  
 weighted avg      0.68        0.67        0.67         100
```



Result:

The SVM model was successfully implemented with RBF kernel and Grid Search. Its performance was evaluated using accuracy, precision, recall, F1-score, confusion matrix and ROC-AUC.

Experiment 4B — Ensemble Methods: Random Forest

Aim

To implement a Random Forest classifier, tune its hyperparameters using Grid Search, evaluate its performance, and identify important features.

Algorithm

1. Import the required libraries.
2. Load the Pet Adoption dataset.
3. Handle missing values.
4. Convert the target into two classes.
5. Encode categorical variables.
6. Split the dataset into training and testing sets.
7. Create a Random Forest classifier.
8. Define the hyperparameter grid.
9. Perform Grid Search with cross-validation.
10. Train the best Random Forest model.
11. Predict the test data.
12. Calculate accuracy, precision, recall, F1-score and ROC-AUC.
13. Display the confusion matrix.
14. Plot the important features and ROC curve.

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.preprocessing import LabelEncoder

from sklearn.ensemble import RandomForestClassifier
```



```

from sklearn.metrics import (

    accuracy_score,

    precision_score,

    recall_score,

    f1_score,

    confusion_matrix,

    classification_report,

    roc_auc_score,

    roc_curve
)

df = pd.read_csv("pet_adoption.csv")

df["Breed"] = df["Breed"].fillna(df["Breed"].mode()[0])

df["Vaccinated"] = df["Vaccinated"].fillna(df["Vaccinated"].mode()[0])

df["Fee"] = df["Fee"].fillna(df["Fee"].median())

df["AdoptionBinary"] = (df["AdoptionSpeed"] <= 2).astype(int)

X = df.drop(["AdoptionSpeed", "AdoptionBinary"], axis=1)

y = df["AdoptionBinary"]

categorical_cols = X.select_dtypes(

    include=["str", "object"]

).columns

X_encoded = X.copy()

for col in categorical_cols:

    le = LabelEncoder()

```

```
X_encoded[col] = le.fit_transform(
    X_encoded[col].astype(str)
)

X_train, X_test, y_train, y_test = train_test_split(
    X_encoded,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

rf = RandomForestClassifier(
    random_state=42,
    n_jobs=-1
)

param_grid = {
    "n_estimators": [100],
    "max_depth": [None, 10],
    "min_samples_split": [2],
    "min_samples_leaf": [1]
}

grid = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
```

```

        scoring="f1",

        cv=3,

        n_jobs=-1

    )

    grid.fit(X_train, y_train)

    print("Best Parameters:", grid.best_params_)

    print("Best Cross-Validation Score:", round(grid.best_score_, 4))

    best_rf = grid.best_estimator_

    y_pred = best_rf.predict(X_test)

    y_prob = best_rf.predict_proba(X_test)[:, 1]

    accuracy = accuracy_score(y_test, y_pred)

    precision = precision_score(y_test, y_pred)

    recall = recall_score(y_test, y_pred)

    f1 = f1_score(y_test, y_pred)

    auc = roc_auc_score(y_test, y_prob)

    cm = confusion_matrix(y_test, y_pred)

    print("\n===== RANDOM FOREST RESULTS =====")

    print("Accuracy :", round(accuracy, 4))

    print("Precision:", round(precision, 4))

    print("Recall  :", round(recall, 4))

    print("F1-Score :", round(f1, 4))

    print("ROC-AUC  :", round(auc, 4))

    print("\nClassification Report:")

```

```
print(classification_report(y_test, y_pred))

plt.figure(figsize=(6, 5))

sns.heatmap(

    cm,

    annot=True,

    fmt="d",

    cmap="Blues",

    cbar=False

)

plt.title("Random Forest Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()

importances = pd.Series(

    best_rf.feature_importances_,

    index=X_encoded.columns

)

top_features = importances.sort_values(

    ascending=False

).head(10)

plt.figure(figsize=(8, 5))

top_features.sort_values().plot(
```

```
        kind="barh"
    )
    plt.xlabel("Importance")
    plt.title("Top Feature Importances - Random Forest")
    plt.tight_layout()
    plt.show()

    fpr, tpr, thresholds = roc_curve(
        y_test,
        y_prob
    )

    plt.figure(figsize=(7, 5))

    plt.plot(
        fpr,
        tpr,
        label=f"Random Forest (AUC = {auc:.3f})"
    )

    plt.plot(
        [0, 1],
        [0, 1],
        linestyle="--"
    )

    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
```

```
plt.title("Random Forest ROC Curve")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

Output:

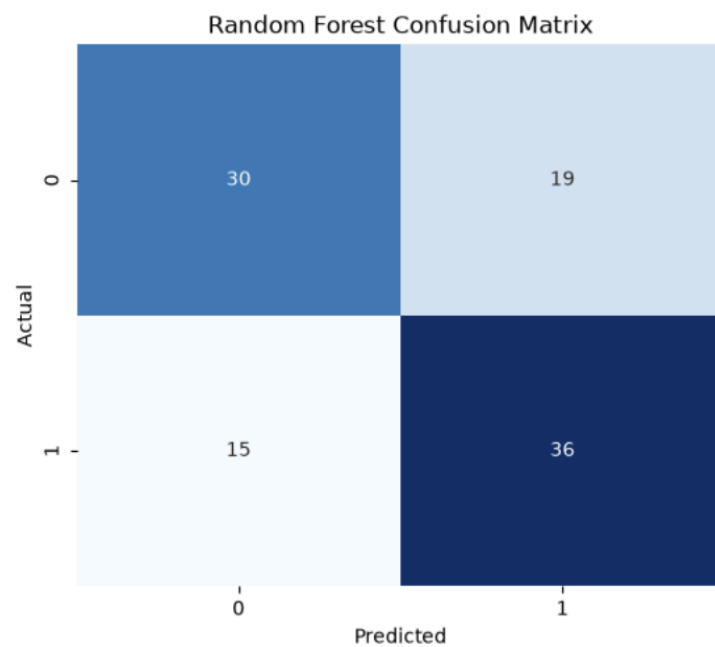
```
Best Parameters: {'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}  
Best Cross-Validation Score: 0.6692
```

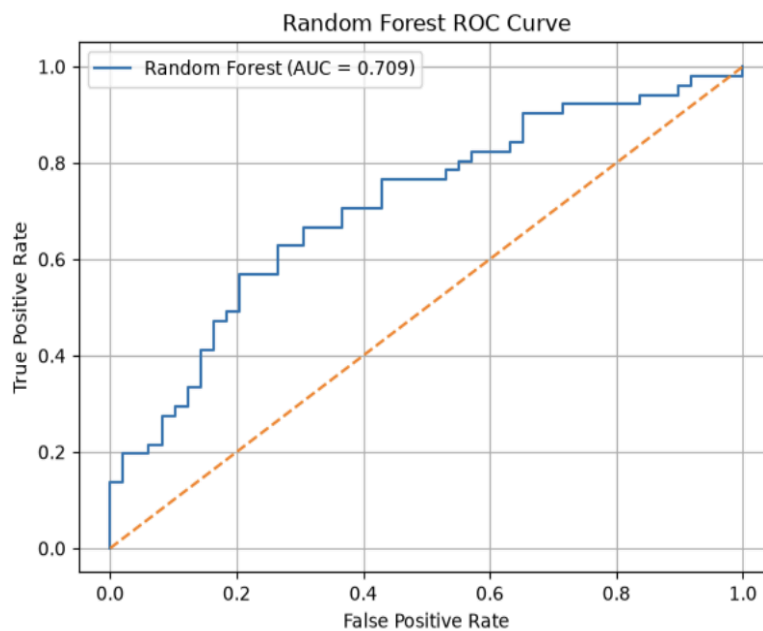
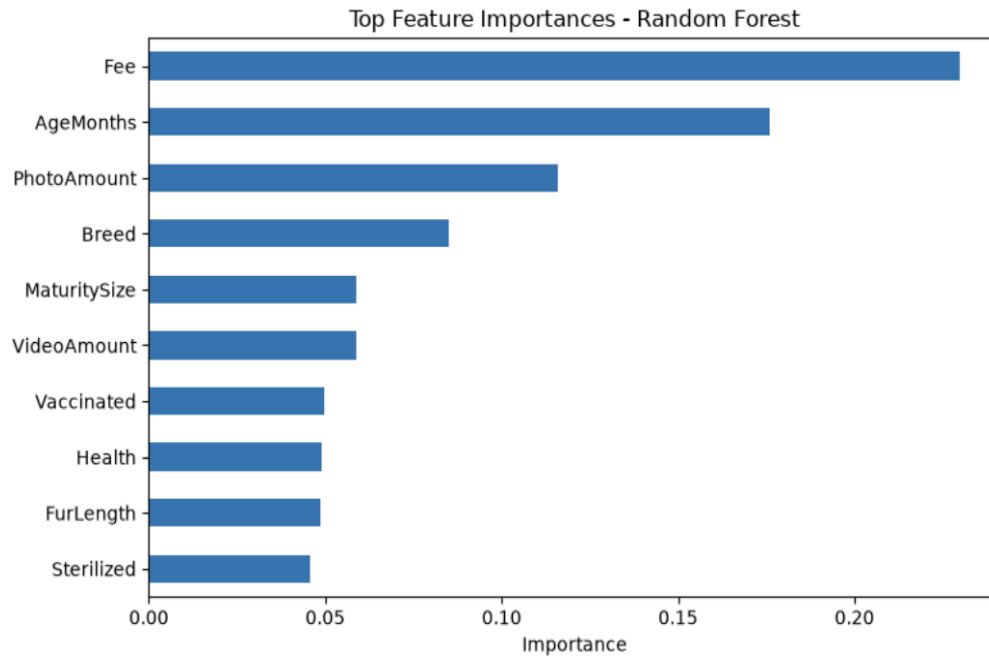
```
===== RANDOM FOREST RESULTS =====
```

```
Accuracy : 0.66  
Precision: 0.6545  
Recall   : 0.7059  
F1-Score : 0.6792  
ROC-AUC  : 0.7095
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.67	0.61	0.64	49
1	0.65	0.71	0.68	51
accuracy			0.66	100
macro avg	0.66	0.66	0.66	100
weighted avg	0.66	0.66	0.66	100





Result:

The Random Forest model was successfully implemented and tuned using Grid Search. Its performance was evaluated using classification metrics, confusion matrix, ROC-AUC and feature importance.